Color Threshold Adjustment and Color Calibration

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the corresponding Dofbot container:

```
./Docker_Ros.sh
```

Then enter the following command in the docker terminal to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then enter the main board's IP address plus :9999 in the browser,

```
IP:9999         // Example: 192.168.1.11:9999
```

By adjusting the high and low thresholds of HSV, filter out interfering colors so that blocks can be ideally recognized in complex environments. When using color-based features for the first time, it's best to calibrate. Each AI feature related to color has its own [HSV_calibration.ipynb] file in the [scripts] folder.

Use the color calibration file at this path for calibration:

/root/dofbot_ws/src/dofbot_color_identify/scripts/HSV校准.ipynb

1. HSV Introduction

HSV (Hue, Saturation, Value) is a color space created by A. R. Smith in 1978 based on the intuitive characteristics of colors, also known as the Hexcone Model. The color parameters in this model are: Hue (H), Saturation (S), and Value (V).

H: 0 — 180

S: 0 — 255

V: 0 — 255

- HSV Parameter Table:

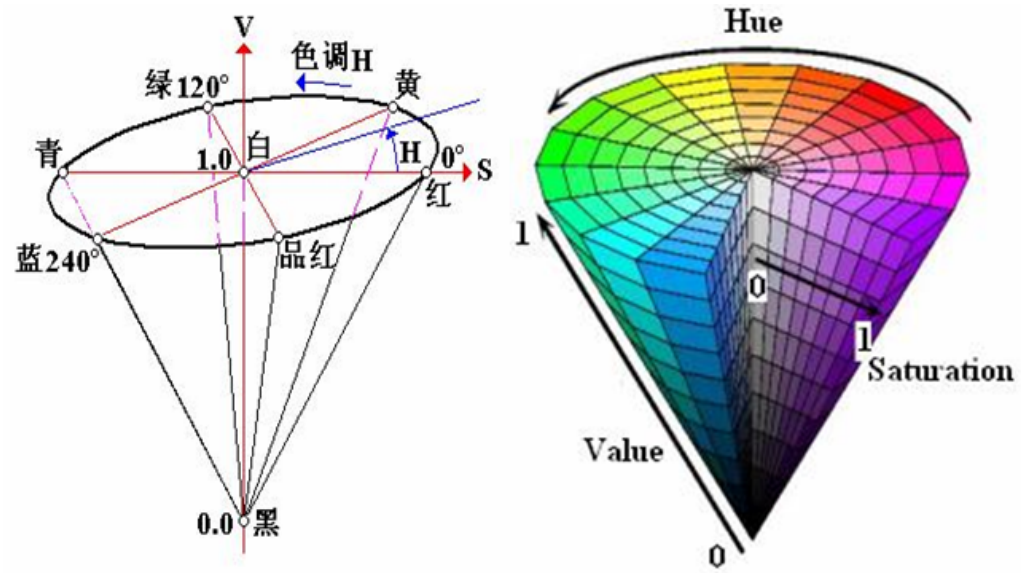
	Black	Gray	White	Red	Orange	Yellow	Green	Cyan	Blue	Purple	
H_min	0	0	0	0	156	11	26	35	78	100	125
H_max	180	180	180	10	180	25	34	77	99	124	155
S_min	0	0	0	43	43	43	43	43	43	43	
S_max	255	43	30	255	255	255	255	255	255	255	
V_min	0	0	0	46	46	46	46	46	46	46	
V_max	46	220	255	255	255	255	255	255	255	255	

- HSV Hexagonal Pyramid

(1) Hue H. Represents color information, i.e., the position of the spectral color. This parameter is represented by an angle measurement, ranging from 0° to 360°, calculated counterclockwise starting from red. Red is 0°, green is 120°, blue is 240°. Their complementary colors are: yellow is 60°, cyan is 180°, purple is 300°.

(2) Saturation S. Saturation S: represents the ratio between the purity of the selected color and the maximum purity of that color. When S=0, there is only grayscale. Complementary colors are 120 degrees apart. Complementary colors differ by 180 degrees. A color can be seen as the result of mixing a certain spectral color with white. The larger the proportion of spectral color, the higher the degree to which the color approaches the spectral color, and the higher the color saturation. High saturation results in deep and bright colors. The white light component of spectral colors is 0, and saturation reaches its maximum. The usual range is 0% to 100%, where larger values indicate more saturated colors.

(3) Value V. Value represents the brightness of the color. For light source colors, the value is related to the brightness of the emitter; for object colors, this value is related to the transmittance or reflectance of the object. The usual range is 0% (black) to 100% (white). One thing to note: there is no direct relationship between it and light intensity. The 3D representation of the HSV model evolved from the RGB cube. Imagine viewing the RGB cube from the white vertex to the black vertex along the diagonal, and you can see the hexagonal appearance of the cube. The hexagonal boundary represents color, the horizontal axis represents purity, and brightness is measured along the vertical axis.



2. Code Design

- Import header files

```
import threading
import cv2 as cv
from time import sleep
from dofbot_config import *
import ipywidgets as widgets
from IPython.display import display
```

- Create instance, initialize parameters

```
# Create HSV update instance
update_hsv = update_hsv()
```

```
# Initialize num parameter
num=0
# Initialize mode
model = "General"
# Initialize HSV name
HSV_name=None
# Initialize HSV value
color_hsv = {"red" : ((0, 143, 163), (11, 255, 255)),

        "green" : ((55, 80, 66), (78, 255, 255)),

        "blue" : ((110, 100, 121), (117, 255, 255)),

        "yellow": ((26, 100, 91), (32, 255, 255))}

# HSV parameter path (jupyter lab and ros function packages need to use absolute
paths)
HSV_path="/root/dofbot_ws/src/dofbot_color_identify/scripts/HSV_config.txt"
# Read HSV configuration file, update HSV value
try: read_HSV(HSV_path,color_hsv)
except Exception: print("No HSV_config file!!!")
```

- Create controls

```
button_layout = widgets.Layout(width='320px', height='55px',
align_self='center')
output = widgets.Output()
# Enter color update mode
HSV_update_red= widgets.Button(description='HSV_update_red',
button_style='success',layout=button_layout)
...
# Adjust slider

H_min_slider=widgets.IntSlider(description='H_min
:',value=0,min=0,max=255,step=1, orientation='horizontal')
...
# Exit
exit_button=widgets.Button(description='Exit',button_style='danger',layout=butto
n_layout)
```

- Color update callback

```
def   update_red_Callback(value):
    pass

def   update_green_Callback(value):
    pass

def   update_blue_Callback(value):
    pass

def   update_yellow_Callback(value):
    pass

HSV_update_red.on_click(update_red_Callback)
HSV_update_green.on_click(update_green_Callback)
```

```
HSV_update_blue.on_click(update_blue_Callback)
HSV_update_yellow.on_click(update_yellow_Callback)
```

- Mode switching controls

```
def   write_file_Callback(value):
    pass

def   Color_Binary_Callback(value):
    pass

def   exit_button_Callback(value):
    pass

HSV_write_file.on_click(write_file_Callback)
Color_Binary.on_click(Color_Binary_Callback)
exit_button.on_click(exit_button_Callback)
```

```
See detailed code at
/root/Dofbot_2D/src/dofbot_color_identify/scripts/HSV_calibration.ipynb
```

- Interface example

The upper part of the screen shows which color is selected. The six sliders in the upper right correspond to the six HSV values. Slide the sliders to adjust the HSV threshold of each color in real time.

3. Operation Process

(1). After starting all code blocks, the interface shown in the figure will be displayed at the bottom of the code. The default color selection is empty, so no color is recognized.

(2). Click the [HSV_update_green] button to start recognizing green objects (Note: this color recognition detects contours, so the object must be completely within the camera range for normal recognition). Slide the sliders in the upper right to adjust the green HSV threshold. When adjusting, pay attention to multi-directional adjustment and adjust in different viewing environments until the object can be clearly recognized in complex environments without interference from other objects [similar for other colors].

(3). The [Color/Binary] button switches between color image and binary image. It is only effective when a color is selected. After switching, only the binary image of the selected color is displayed, making it more convenient for us to debug.

(4). The [HSV_write_file] button. After debugging the HSV values of all colors, click this button. Save all debugged parameters in [.txt] format in the same path as the code. The file parameters will be automatically read on the next startup.

(5). The [Exit] button closes the camera and exits the program.